



Analítica de Big Data con Spark

82.81

Entrega TP2

Sistema de recomendación

Profesores:

Nicolas Leonel Santilli

Grupo - Legajos:

Lara Pascaretta - 64776

Agustina Kohan Miller - 64231

Stefania Ranucci - 63694

Fecha de entrega:

18/06/2026

1. Elección del dataset y estrategia de ejecución

Elegimos hacer un sistema de recomendación de música ya que a las tres nos interesa la música pero nos vemos limitadas a la hora de explorar nuevo contenido por las sugerencias poco innovadoras de Spotify. Notamos que dentro de Spotify, el sistema de recomendación parece enfocarse en replicar el contenido más escuchado, en vez de buscar expandir los horizontes del usuario fuera de su zona de confort.

Inicialmente usamos el dataset HetRec 2011 del link de la consigna, pero al analizarlo vimos que era significativamente más chico de lo esperado (1,892 usuarios, 92k interacciones), por lo que cambiamos al Last.fm 360K (upf.edu/web/mtg/lastfm360k). La fuente original de GitHub dejó de estar disponible, así que usamos un mirror de Internet Archive. Incorporamos la descarga automática directamente en la notebook para que sea reproducible sin descargar nada a mano. El dataset tiene 359.349 usuarios únicos, 160.168 artistas únicos y 17.559.530 interacciones totales.

En cuanto al subconjunto elegido, el dataset completo no es procesable en Colab. Filtramos en dos pasos: En primer lugar, para los usuarios mantenemos los que tienen entre 10 y 500 reproducciones totales. El umbral inferior elimina usuarios con historial insuficiente para generar recomendaciones útiles o para hacer un train/test split significativo. El umbral superior elimina usuarios con comportamiento atípico que podría sesgar los embeddings de ALS. En segundo lugar, para los artistas mantenemos los que tienen al menos 50 oyentes únicos, eliminando la cola derecha que introduce ruido sin sumar señal.

Todo el proyecto fue desarrollado y ejecutado inicialmente en Google Colab (Python 3.12, PySpark 4.0.2, single node). Para las iteraciones de mejora posteriores migramos a Kaggle, que ofrece más RAM y sesiones más largas, ya que en Colab el pipeline completo tardaba demasiado y las desconexiones nos hacían perder trabajo. El pipeline corre igual en ambos entornos. Como ninguno de los dos persiste archivos entre sesiones, guardamos el dataset procesado en formato Parquet para poder recargarlo en celdas posteriores sin recomputar todo (especialmente útil dado que el filtrado y el StringIndexer son las etapas más costosas del pipeline). Intentamos usar Databricks inicialmente, pero la versión gratuita solo ofrece cómputo Serverless, que no permite usar pyspark.ml ni entrenar ALS.

2. Exploración y análisis con PySpark

El dataset utilizado corresponde a un sistema de recomendación musical basado en interacciones implícitas. La variable principal es weight, que representa la cantidad de reproducciones de un artista por parte de un usuario. Por lo tanto, una mayor cantidad de escuchas se interpreta como una señal de mayor afinidad, aunque la ausencia de interacción no implica necesariamente una preferencia negativa.

La distribución de weight presenta una fuerte asimetría positiva: la mediana es de 94 reproducciones, el tercer cuartil de 224 y el máximo alcanza 419.157. Esto indica que la mayoría de las interacciones son moderadas, pero existen pocos casos con volúmenes de escucha extremadamente altos.

La matriz usuario-ítem es altamente dispersa. En el dataset completo, la densidad es de 0,0305%, lo que significa que sólo una proporción mínima de las combinaciones posibles usuario-artista está observada. Luego del filtrado aplicado, el subconjunto final quedó compuesto por 22.125 usuarios, 25.128 artistas y 809.540 interacciones, con una densidad de 0,15%. Si bien el filtrado aumenta la densidad, la matriz sigue siendo muy sparse, como es esperable en sistemas de recomendación de este estilo, ya que con tantos artistas es casi imposible que alguien conozca y escuche todos.

Respecto a las distribuciones:

- Interacciones por usuario: la mayoría de los usuarios escucha una cantidad intermedia de artistas, con un pico cercano a los 50 artistas únicos. También se observa una cola de usuarios con una actividad mucho mayor.
- Reproducciones por usuario: predominan usuarios con volúmenes bajos o medios de plays, mientras que pocos usuarios concentran cantidades muy elevadas de escuchas.
- Interacciones por artista: la concentración es más marcada. La mayoría de los artistas tiene pocos oyentes y pocas reproducciones, mientras que un grupo reducido concentra gran parte de la actividad.

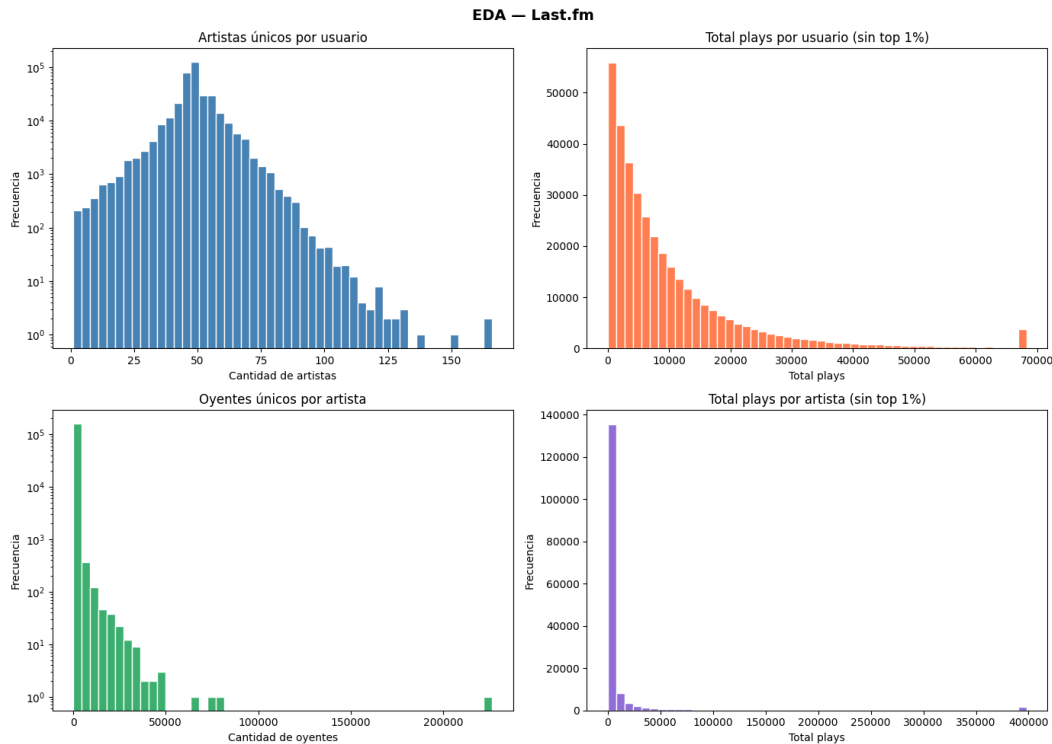


Figura 1. Distribuciones exploratorias del dataset Last.fm

Una observación no obvia encontrada es la concentración del poder: el top 1% de artistas (1601 distintos) concentran el 64.1% de todas las escuchas. Esto pone en evidencia un fuerte sesgo de popularidad, con pocos artistas masivos y una larga cola de artistas de nicho. Lo cual es representativo de la vida real, ya que es más fácil encontrar a un artista popular por recomendación boca a boca. Esta característica justifica el uso de ALS con `implicitPrefs=True`, ya que el modelo trata las reproducciones como señales implícitas de confianza y no como ratings explícitos. Así, muchas escuchas refuerzan la preferencia estimada, pero la falta de interacción no se interpreta automáticamente como una valoración negativa.

3. Estrategia de ejecución con recursos limitados

El dataset completo, como ya mencionamos, tiene 17.5M de interacciones, por lo que no entra en memoria de un solo nodo. El problema más común en recomendación es el negative sampling via `crossJoin` (cruzar todos los usuarios con todos los artistas generaría 57 mil millones de pares en nuestro caso). Lo evitamos usando ALS con `implicitPrefs=True`, que trata las interacciones no observadas como negativos implícitos sin materializar la matriz completa.

Para el resto del pipeline, cacheamos `train_df` antes de entrenar para evitar recomputación en cada iteración de ALS, y persistimos el dataset procesado en un formato Parquet para no repetir las

etapas más costosas si la sesión se cae. Usamos `toPandas()` solo para visualizaciones sobre datos ya agregados, nunca sobre el dataset completo.

La `SparkSession` se configuró con `driver.memory=4g`, `shuffle.partitions=200` y `autoBroadcastJoinThreshold=-1` para evitar OOM en joins de features.

En cuanto a sesgos: el filtro 10–500 artistas retiene solo el 6.2% de usuarios, priorizando actividad media y excluyendo usuarios con poco historial. Esto hace que nuestras métricas @K sean probablemente más altas de lo que serían sobre el dataset completo (los usuarios difíciles de recomendar quedaron fuera del subconjunto). Adicionalmente, dado que el dataset Last.fm 360K no incluye timestamps en este archivo, el split train/test se hizo de forma aleatoria por usuario. Cabe aclarar que en un entorno productivo este split debería ser estrictamente cronológico para evitar predecir el pasado con el futuro (data leakage temporal).

4. Preprocesamiento con PySpark

Todo el preprocesamiento del dataset se realizó en PySpark, evitando cargar el conjunto completo en memoria local. El objetivo fue transformar las interacciones originales de Last.fm en una matriz usuario-artista apta para entrenar un modelo ALS con feedback implícito.

Primero se aplicó un filtrado del dataset para trabajar con un subconjunto manejable y representativo. Se conservaron usuarios con entre 10 y 500 reproducciones totales, excluyendo tanto usuarios con muy poca actividad como power users extremos. Además, se conservaron artistas con al menos 50 oyentes únicos, para reducir la cola larga de artistas con muy poca señal colaborativa. Luego del filtrado, el subconjunto final quedó compuesto por 22.125 usuarios, 25.128 artistas y 809.540 interacciones.

Re-indexación con StringIndexer

ALS requiere que los identificadores de usuarios e ítems sean numéricos y enteros. Por eso se usó `StringIndexer` para transformar los IDs originales en índices consecutivos:

- `userID` → `user_index`
- `artistID` → `artist_index`

El ajuste de los indexadores se hizo sobre el subconjunto ya filtrado, no sobre el dataset completo, para reducir memoria y tiempo de procesamiento. Luego, las columnas fueron casteadas a los tipos requeridos por ALS: índices como `IntegerType` y la variable de interacción como `FloatType`.

- Features adicionales de usuario e ítem

Aunque ALS no incorpora features adicionales directamente en el entrenamiento, se calcularon variables complementarias en PySpark para caracterizar mejor el dataset y dejar preparado el pipeline para modelos más expresivos, como Two-Tower o modelos híbridos.

Para los usuarios se construyeron las siguientes features:

- `log_total_plays`: logaritmo del total de reproducciones del usuario.
- `n_unique_artists`: cantidad de artistas distintos escuchados.
- `gender`: género del usuario, tomado del archivo de perfiles.
- `country`: país del usuario, también tomado del perfil.

Para los artistas se construyeron:

- `log_artist_popularity`: logaritmo del total de reproducciones recibidas por el artista.
- `n_unique_listeners`: cantidad de usuarios únicos que escucharon al artista.

Estas features combinan información de comportamiento, como volumen de escucha y diversidad musical, con información demográfica disponible en el dataset.

Normalización y codificación

Las variables de conteo más asimétricas fueron transformadas usando $\log_{1p}$, especialmente `total_plays` y la popularidad total de los artistas. Esta transformación reduce el impacto de valores extremos y permite representar mejor distribuciones altamente sesgadas, como las observadas en el análisis exploratorio.

Las variables categóricas `gender` y `country` fueron tratadas como texto y los valores faltantes se completaron con "unknown". Dado que el modelo final fue ALS, estas variables no ingresaron directamente al entrenamiento. Sin embargo, en un modelo Two-Tower o híbrido podrían codificarse mediante embeddings categóricos u one-hot encoding.

Finalmente, el dataset procesado fue guardado en formato Parquet para evitar recomputar todo el pipeline en cada sesión. Esto permitió mejorar la eficiencia del trabajo en Colab y facilitar la reproducibilidad del notebook.

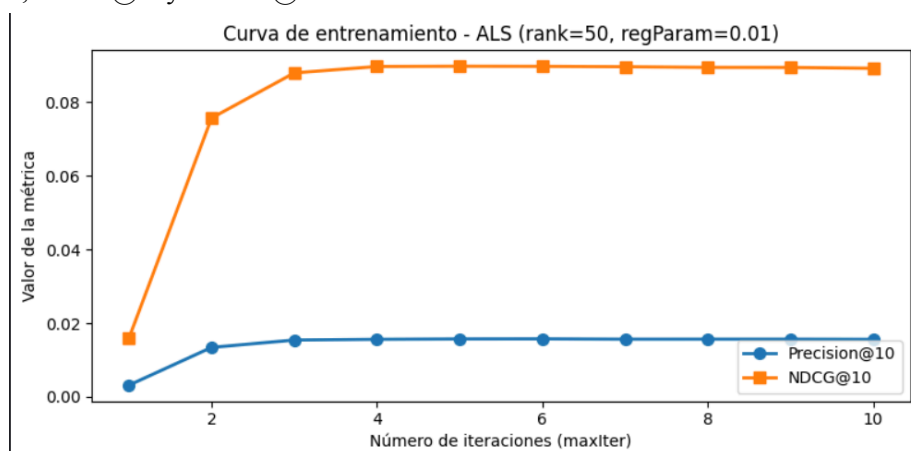
5. Elección del modelo y entrenamiento

Elegimos ALS por dos razones principales. Primero, es el modelo más adecuado para datos implícitos como los nuestros, ya que Last.fm registra cantidades de reproducciones, no ratings, y ALS con `implicitPrefs=True` fue diseñado específicamente para este caso: trata la cantidad de reproducciones como señal de confianza en lugar de una preferencia directa. Segundo, corre 100% en Spark sin depender de PyTorch ni GPU, lo que lo hace ideal para Colab sin GPU garantizada.

Para la experimentación de hiper parámetros evaluamos cuatro configuraciones combinando $\text{rank} \in \{10, 50\}$ y $\text{regParam} \in \{0.01, 0.1\}$:

rank	regParam	RMSE	Precision@10	Recall@10	NDCG@10
50	0.01	12.2414	0.0156	0.1562	0.0892
50	0.10	12.2453	0.0156	0.1557	0.0888
10	0.01	12.2991	0.0096	0.0965	0.0574
10	0.10	12.3011	0.0096	0.0964	0.0566

La mejor configuración fue `rank=50, regParam=0.01`. Vale aclarar que con `implicitPrefs=True` el RMSE no es interpretable en valor absoluto (ALS produce scores de confianza internos, no play counts). Estos valores solo sirven para comparar configuraciones entre sí; las métricas relevantes son `Precision@K`, `Recall@K` y `NDCG@K`.



Curva de entrenamiento: medimos `Precision@10` y `NDCG@10` por iteración, que convergió prácticamente en la iteración 4-5 (mejora relativa < 1%), con `NDCG@10` máximo de 0.0898 en

iteración 5. Las métricas de ranking sobre el conjunto de test (K=10, 21.810 usuarios) fueron: Precision@10: 0.0156 - Recall@10: 0.1562 - NDCG@10: 0.0892.

Los resultados tienen coherencia cualitativa. El usuario que escucha vienna teng, fito y fitipaldi y extremoduro recibe recomendaciones de estopa, jarabe de palo, ismael serrano y melendi, manteniéndose en el género de pop/rock español. El usuario de beatles, a.r. rahman y kishore kumar recibe recomendaciones que incluyen nusrat fateh ali khan y artistas de world music, capturando su gusto por música clásica y fusión. El tercero, que escucha julieta venegas, russian red y joaquín sabina, recibe recomendaciones como la oreja de van gogh, gilberto gil y gal costa, artistas de rock y música mundial afines. En los tres casos el modelo capturó correctamente el perfil del usuario sin recomendar artistas ya escuchados en entrenamiento.

ALS no puede incorporar las features adicionales calculadas directamente en el entrenamiento. La alternativa más directa sería un modelo híbrido que use los embeddings de ALS como input de un segundo modelo, o Two-Tower que combina features en cada torre nativamente.

Como comparación adicional, implementamos BPR (Bayesian Personalized Ranking) sobre el mismo subconjunto y split. ALS superó a BPR en todas las métricas, demostrando que la factorización matricial con implicitPrefs es más adecuada para este dataset que el ranking explícito de BPR, con la ventaja adicional de correr nativamente en Spark.

Modelo	Precision@10	Recall@10	NDCG@10
ALS (rank=50, regParam=0.01)	0.0156	0.1562	0.0892
BPR (emb_dim=32, 25 épocas)	0.0092	0.0917	0.0524

6. Sección de escalabilidad

A una escala de 100 millones de interacciones, el pipeline debería mantenerse completamente distribuido. La generación de negativos no podría hacerse con un crossJoin, porque produciría una cantidad inmanejable de pares usuario-artista. En su lugar, se usaría negative sampling distribuido, generando pocos negativos por interacción positiva, idealmente combinando muestreo aleatorio, por popularidad y hard negatives.

También dejaría de ser viable usar toPandas(), ya que implicaría mover grandes volúmenes de datos a la memoria del driver. En su reemplazo, los datos deberían permanecer en Spark y persistirse en formatos como Parquet o Delta Lake, trabajando siempre por particiones y batches.

Si se usara un modelo neuronal como Two-Tower o BPR, el entrenamiento podría distribuirse con TorchDistributor en Databricks, utilizando Spark para el preprocesamiento y PyTorch distribuido para entrenar embeddings en varios workers con GPU.

Para servir recomendaciones en tiempo real, se precomputarían diariamente recomendaciones top-K por usuario y se guardarían en una base de baja latencia, como Redis, Cassandra o una tabla Delta optimizada. Para casos más dinámicos, se podría combinar este ranking batch con búsqueda aproximada sobre embeddings y un re-ranking rápido según señales recientes del usuario.

7. Extra

Como bonus, dado que una de las integrantes del grupo (Lara) usa Last.fm hace años, nos pareció interesante probar si podíamos cargar sus datos reales y generar recomendaciones personalizadas. Para esto generamos una API key gratuita desde last.fm/api y usamos el endpoint user.getTopArtists para descargar el top 200 artistas con los play counts reales. Luego cruzamos esos artistas con los del dataset 360K por nombre (normalizado a minúsculas) para ver cuáles el modelo ya conocía.

De los 200 artistas más escuchados, solo 83 aparecen en el dataset (los 117 restantes no están porque son más nuevos que los datos o muy de nicho). Para poder recomendar algo, tuvimos que resolver el problema del cold start: ALS aprende un vector latente por usuario durante el entrenamiento, así que un usuario nuevo no tiene representación en el modelo. La solución fue agregar las interacciones del usuario nuevo al dataset original y reentrenar ALS con esos datos incorporados (17,559,613 interacciones totales + 83 de Lara). Con el modelo reentrenado, generamos dos listas: artistas que nunca escuchó pero el modelo predice que le van a gustar, y "redescubrimientos", que son artistas que escuchó muy poco y el modelo cree que merece darles otra oportunidad. Finalmente, usamos la API de Last.fm para traer la canción más popular de cada artista recomendado.